Riemannian Sheaf Neural Networks

Federico Barbero *
University of Cambridge
fb548@cam.ac.uk

Cristian Bodnar
University of Cambridge
cb2015@cam.ac.uk

Abstract

A Sheaf Neural Network (SNN) is a type of Graph Neural Network (GNN) which operates over a sheaf, an object which equips a graph with vector spaces over the nodes and edges and linear maps between them. This model has been shown to have useful theoretical properties that tackle issues arising from heterophily and over smoothing. One complication that comes with this new paradigm is computing the sheaf. In general, the ground truth sheaf is unknown, and SNNs learn this structure from the data via standard gradient-based approaches. In this work, we propose a novel way of computing a sheaf directly from the data by leveraging intuition from Riemannian geometry. We leverage the manifold assumption to compute manifold-and-graph aware linear maps, which optimally align the local tangent spaces of each data point by orthogonal maps. We show that this approach achieves promising results on a much more limited amount of fine-tuning when compared to original SNN models. Overall, this work provides an interesting connection between algebraic topology and differential geometry, and we hope that it will spark future research in this direction.

1 Introduction

Graph Neural Networks (GNNs) (Scarselli et al. [23]) have shown encouraging results in a wide range of applications, ranging from drug design to guiding discoveries in pure mathematics (Davies et al. [10]). One advantage over traditional Neural Networks is that they can leverage the extra structure in graph data, namely the edge connections.

GNNs, however, do not come without issues. Traditional GNN models, such as Graph Convolutional Networks (GCNs) (Kipf and Welling [17]) have been shown to work poorly on data that has a low level of homophily (in other words, heterophilic), i.e., when connected nodes tend to not belong to the same class or to have dissimilar feature vectors. For the most part, this issue comes by design, as, for example, in GCNs, homophily may be seen as an inductive bias of the model. Another problem that comes with GNNs is the phenomenon of over smoothing (Oono and Suzuki [20]). GNNs tend not to improve or even worsen as more layers are added. These two problems are, from a geometric point of view, intimately connected (Chen et al. [6], Bodnar et al. [3]).

Bodnar et al. [3] have shown that when the underlying "geometry" of the graph is too simple, the issues discussed above arise. More precisely, they analyse the geometry of the graph through cellular sheaf theory (Curry [9], Hansen [12], Hansen and Ghrist [14]), a subfield of algebraic topology (Hatcher [16]). A cellular sheaf over a graph associates vector spaces to edges and nodes and maps between them. A graph neural network which operates over a cellular sheaf is known as a Sheaf Neural Network (SNN) (Hansen and Gebhart [13], Bodnar et al. [3]).

*Report for the L45: Representation Learning on Graphs and Networks Mini-Project.

SNNs work by computing a Sheaf Laplacian, and it can be shown that when the underlying sheaf is "trivial" (with 1-dimensional vector spaces and identity maps between them), one recovers the well-known graph Laplacian. Using higher dimensional vector spaces has theoretical advantages but unfortunately comes at the cost of a higher-dimensional sheaf Laplacian. Bodnar et al. [3] propose to *learn* this sheaf Laplacian from the data, which can lead to computational complexity problems and overfitting.

This work proposes a novel technique that aims to compute the sheaf Laplacian directly from the data in a deterministic manner, removing the need to learn it with gradient-based approaches. We do this through the lens of differential geometry by assuming that the data is sampled from a low dimensional manifold and optimally aligning the bases of tangent spaces by orthogonal transformations. This idea was first introduced as groundwork for Vector Diffusion Maps by Singer and Wu [24]; however, it only assumed a point-cloud structure. Instead, one of our contributions involves the computation of these optimal alignments over a graph structure.

In section 2, we give a brief overview of cellular sheaf theory and of neural sheaf diffusion (Bodnar et al. [3]). Next, in section 3, we give details of our new procedure used to compute the sheaf Laplacian deterministically, giving rise to Riemannian Sheaf Diffusion (Riem-SD). We then, in section 4, evaluate this technique on various datasets with ranging homophily levels. Our evaluation also includes an exploration of the structure of the computed sheaves. We believe that this work is a promising attempt at connecting algebraic topology and differential geometry methods applied to machine learning and hope that this may push more work towards the study of how to compute sheaf Laplacians best.

2 Background

We briefly overview the necessary background, starting with cellular sheaf theory and concluding with neural sheaf diffusion. The curious reader is directed to work by Curry [9], Hansen [12], Hansen and Ghrist [14] for a more exhaustive overview of cellular sheaf theory and to Bodnar et al. [3] for the full theoretical results of neural sheaf diffusion.

2.1 Cellular Sheaf Theory

We start by giving a brief overview of cellular sheaves.

Definition 2.1. A cellular sheaf $(\mathcal{G}, \mathcal{F})$ on an *undirected graph* $G = (V, E)$ consists of:

- A vector space $\mathcal{F}(v)$ for each $v \in V$.
- A vector space $\mathcal{F}(e)$ for each $e \in E$.
- A linear map $\mathcal{F}_{v \trianglelefteq e} : \mathcal{F}(v) \to \mathcal{F}(e)$ for each incident node-edge pair $v \trianglelefteq e$.

The vector spaces of the node and edges are called *stalks*, while the linear maps are called *restriction maps*. It is then natural to group the various spaces. The space which is formed by the node stalks is called the space of 0-cochains, while the space formed by edge stalks is called the space of 1-cochains. A diagrammatic representation of this structure for a single edge may be found in figure 1, courtesy of Bodnar et al. [3].

Definition 2.2. Given a sheaf $(\mathcal{G}, \mathcal{F})$, we define the space of 0-cochains $C^0(\mathcal{G}, \mathcal{F})$ as the direct sum over the vertex stalks $C^0(\mathcal{G}, \mathcal{F}) := \bigoplus_{v \in V} \mathcal{F}(v)$. Similarly, the space of 1-cochains $C^1(\mathcal{G}, \mathcal{F})$ as the direct sum over the edge stalks $C^1(\mathcal{G}, \mathcal{F}) := \bigoplus_{e \in E} \mathcal{F}(e)$.

Defining the spaces $C^0(\mathcal{G}, \mathcal{F})$ and $C^1(\mathcal{G}, \mathcal{F})$ allows us to construct a linear *co-boundary map* $\delta : C^0(\mathcal{G}, \mathcal{F}) \to C^1(\mathcal{G}, \mathcal{F})$. From an opinion dynamics perspective (Hansen and Ghrist [15]), the node stalks may be thought of as the private space of opinions and the edge stalks as the space in which these opinions are shared in a public discourse space. The co-boundary map δ then measures the disagreement between all the nodes.

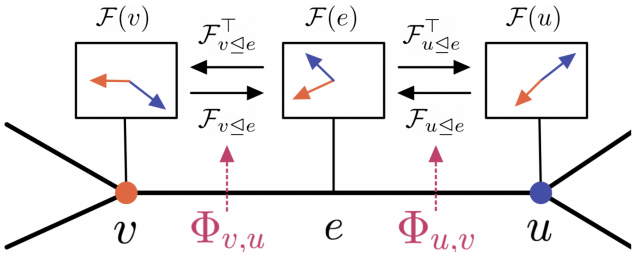

Figure 1: A diagrammatic representation of a sheaf $(G, \mathcal{F})$ for a single edge. The node *stalks* $\mathcal{F}(v)$ and $\mathcal{F}(u)$ and the edge stalk $\mathcal{F}(e)$ are isomorphic to $\mathbb{R}^2$. We can transport features between stalks via the *restriction maps* $\mathcal{F}_{v\trianglelefteq e}$ and $\mathcal{F}_{u\trianglelefteq e}$ and their adjoints $\mathcal{F}_{v\trianglelefteq e}^T$ and $\mathcal{F}_{u\trianglelefteq e}^T$. The parametric function Φ is used to learn the restriction maps from the data. Diagram courtesy of Bodnar et al. [3].

Definition 2.3. Given some arbitrary orientation for each edge $e = u \to v \in E$, we define the co-boundary map $\delta : C^0(\mathcal{G}, \mathcal{F}) \to C^1(\mathcal{G}, \mathcal{F})$ as $\delta(\mathbf{x})_e = \mathcal{F}_{v\trianglelefteq e}\mathbf{x}_v - \mathcal{F}_{u\trianglelefteq e}\mathbf{x}_u$. Here $\mathbf{x} \in C^0(\mathcal{G}, \mathcal{F})$ is a 0-cochain and $\mathbf{x}_v \in \mathcal{F}(v)$ is the vector of $\mathbf{x}$ at the node stalk $\mathcal{F}(v)$.

The co-boundary map δ allows us to construct the *sheaf Laplacian operator* over a sheaf.

Definition 2.4. The Sheaf Laplacian of a sheaf is a map $L_{\mathcal{F}} : C^0(\mathcal{G}, \mathcal{F}) \to C^0(\mathcal{G}, \mathcal{F})$ defined as $L_{\mathcal{F}} = \delta^T\delta$.

The Sheaf Laplacian is a symmetric positive semi-definite (by construction) block matrix. The diagonal blocks are $L_{\mathcal{F}_{v,v}} = \sum_{v\trianglelefteq e} \mathcal{F}_{v\trianglelefteq e}^T \mathcal{F}_{v\trianglelefteq e}$, while the off-diagonal blocks are $L_{\mathcal{F}_{v,u}} = -\mathcal{F}_{v\trianglelefteq e}^T \mathcal{F}_{u\trianglelefteq e}$.

Definition 2.5. The *normalised Sheaf Laplacian* $\Delta_{\mathcal{F}}$ is defined as $\Delta_{\mathcal{F}} = D^{-\frac{1}{2}} L_{\mathcal{F}} D^{-\frac{1}{2}}$ where D is the block-diagonal of $L_{\mathcal{F}}$.

While the dimension of the stalks is arbitrary, we make the simplification to work with node and edge stalks which are all d-dimensional. This means that each restriction map is $d \times d$, and therefore so is each block in the Sheaf Laplacian. With n we denote the number of nodes in the underlying graph G. This means that our Sheaf Laplacian has a dimension of $nd \times nd$.

If we construct a trivial sheaf where each stalk is isomorphic to $\mathbb{R}$ and the restriction maps are identity maps, then we recover the well-known $n \times n$ graph Laplacian from the Sheaf Laplacian. This suggests that the Sheaf Laplacian generalises the graph Laplacian by considering a non-trivial sheaf on G.

Definition 2.6. The orthogonal (Lie) group in dimension d, denoted $O(d)$, is the group of $d \times d$ orthogonal matrices where the group operation is matrix multiplication.

If we restrict the restriction maps in the sheaf to belong to the orthogonal group, i.e., $\mathcal{F}_{v\trianglelefteq e} \in O(d)$, then our sheaf is called a *discrete* $O(d)$*-bundle*. This is because it may be thought of as a discretised version of a vector bundle. The sheaf Laplacian of the $O(d)$-bundle is equivalent to a connection Laplacian (Singer and Wu [24]). The restriction maps, being orthogonal, describe how vectors are rotated when transported between stalks. This process is analogous to the transportation of tangent vectors on a manifold. Figure 2 shows this in more detail, courtesy of Bodnar et al. [3]. We will be using this manifold construction to compute our deterministic sheaf Laplacian.

These restriction maps are advantageous because orthogonal matrices have fewer free parameters, making them more efficient to work with. For example, the underlying manifold of the Lie group $O(d)$ is $\frac{d(d-1)}{2}$-dimensional instead of d^2-dimensional. This can be seen, for

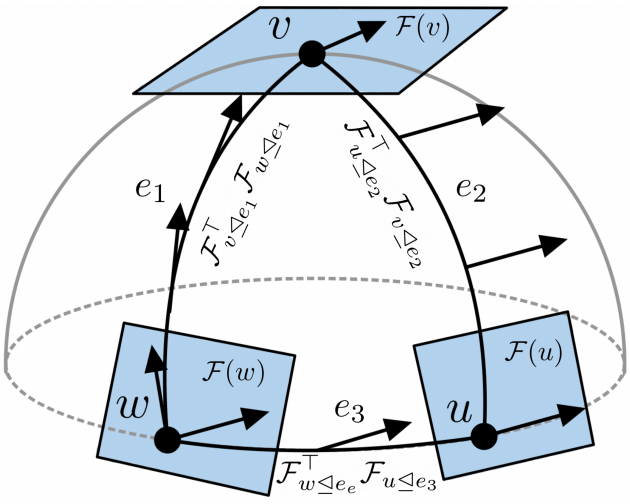

Figure 2: Transport over a discrete vector bundle. Starting at $\mathcal{F}(w)$, we transport the vector to $\mathcal{F}(v)$ by $\mathcal{F}_{v \trianglelefteq e_1}^{T} \mathcal{F}_{w \trianglelefteq e_1}$ and similary to $\mathcal{F}(u)$ and back. Since the resulting vectors have a different final angle, the transport in this case is not path independent. Diagram courtesy of Bodnar et al. [3].

example, from matrices in $O(2)$ which, although they are 2×2, they only need $\frac{2(2-1)}{2} = 1$ free parameter, in the form of a rotation angle.

2.2 Neural Sheaf Diffusion

We will now discuss the existing sheaf-based machine learning models and their theoretical properties. Consider a graph $G = (V, E)$ where each node $v \in V$ has a d-dimensional feature vector $\mathbf{x}_v \in \mathcal{F}(v)$. We can now construct $\mathbf{x} \in C^0(\mathcal{G}, \mathcal{F})$ by stacking the individual vectors $\mathbf{x}_v$, creating a nd-dimensional vector. Allowing for f feature channels, we can construct the feature matrix $\mathbf{X} \in \mathbb{R}^{(nd) \times f}$. Each column of this matrix is a vector in $C^0(\mathcal{G}, \mathcal{F})$ for one of the f channels.

Definition 2.7. We define a spatially discretised *sheaf diffusion* via the following partial differential equation:

$$\mathbf{X}(0) = \mathbf{X}, \dot{\mathbf{X}}(t) = -\Delta_{\mathcal{F}} \mathbf{X}(t) \tag{1}$$

which has the following Euler discretisation with unit step-size:

$$\mathbf{X}(t+1) = \mathbf{X}(t) - \Delta_{\mathcal{F}} \mathbf{X}(t) = (\mathbf{I}_{nd} - \Delta_{\mathcal{F}})\, \mathbf{X}(t)$$

We now discuss results from Bodnar et al. [3] related to the separation power of the sheaf diffusion model.

Definition 2.8. A hypothesis class of sheaves with d-dimensional stalks $\mathcal{H}^d$ has linear separation power over a set of graphs $\mathcal{G}$ if for every graph $G \in \mathcal{G}$ there is a sheaf in $(\mathcal{F}, G) \in \mathcal{H}^d$ that can linearly separate the classes of G in the time limit of the sheaf diffusion equation for a dense subset $\mathcal{X}_{\mathcal{F}}$ of initial conditions.

In the time limit, the diffusion process behaves like a projection into the harmonic space $\ker(L_{\mathcal{F}})$. We now define various types of hypothesis classes.

Definition 2.9. The hypothesis class of sheaves with symmetric and invertible transport maps.

$$\mathcal{H}^d_{\text{sym}} = \{(\mathcal{F}, \mathcal{G}) | \mathcal{F}_{v \trianglelefteq e} = \mathcal{F}_{u \trianglelefteq e}, \det(\mathcal{F}_{v \trianglelefteq e}) \neq 0\}$$

The hypothesis class of sheaves with invertible maps.

$$\mathcal{H}^d = \{(\mathcal{F}, \mathcal{G}) | \det(\mathcal{F}_{v \trianglelefteq e}) \neq 0\}$$

The hypothesis class of sheaves with diagonal invertible maps.

$$\mathcal{H}^d_{\text{diag}} = \{(\mathcal{F}, \mathcal{G}) | \mathcal{F}_{v \trianglelefteq e} = \text{invertible diagonal matrix}\}$$

The hypothesis class of sheaves with orthogonal maps, also known as the hypothesis class of discrete $O(d)$-bundles.

$$\mathcal{H}^d_{\text{orth}} = \{(\mathcal{F}, \mathcal{G}) | \mathcal{F}_{v \trianglelefteq e} \in O(d)\}$$

Bodnar et al. [3] prove various results for these different classes of sheaves, of which we give a summary. $\mathcal{H}^1_{\text{sym}}$ is perhaps the most familiar class as for when $d = 1$ we retrieve the weighted graph Laplacian with strictly positive weights. While this hypothesis class can work well with homophilic graphs, Bodnar et al. [3] show that this class is not powerful enough to separate a graph under specific heterophilic conditions linearly. Instead the larger class $\mathcal{H}^1$ (note that $\mathcal{H}^1_{\text{sym}} \subset \mathcal{H}^1$) is able to still separate under those specific conditions. This points to the observation that asymmetric transport is necessary under heterophily. Another important observation is that $\mathcal{H}^1$ is insufficient when the nodes belong to more than 2 classes; instead, larger stalk width d is necessary. These results tell us that the transport map structure is critical for linear separability and secondly that stalk width also plays a key role when more classes are added. Notably, $\mathcal{H}^d_{\text{orth}}$ has useful theoretical results as it was shown by Bodnar et al. [3] that orthogonal maps are able to make more efficient use of the stalk dimension than the other classes. This is the reason for which we develop a novel technique around orthogonal maps.

Bodnar et al. [3] propose to equip equation 1 with weight matrices $\mathbf{W}^t_1$ and $\mathbf{W}^t_2$. The restriction maps which are used to compute the sheaf $\Delta_{\mathcal{F}(t)}$ are computed by a parametric function $\Phi : \mathbb{R}^{d \times 2} \to \mathbb{R}^{d \times d}$ such that $\mathcal{F}_{v \trianglelefteq e:=(v,u)} = \Phi(\mathbf{x}_v, \mathbf{x}_u)$. Φ may enforce specific structure, such as $\mathcal{F}_{v \trianglelefteq e:=(v,u)}$ being diagonal, orthogonal or general.

$$\dot{\mathbf{X}} = -\sigma\left(\Delta_{\mathcal{F}(t)} \left(I_n \otimes \mathbf{W}_1\right) \mathbf{X}(t) \mathbf{W}_2\right) \tag{2}$$

Equation 2 can be discretised as follows.

$$\mathbf{X}_{t+1} = \mathbf{X}_t - \sigma\left(\Delta_{\mathcal{F}(t)} \left(I_n \otimes \mathbf{W}^t_1\right) \mathbf{X}_t \mathbf{W}^t_2\right) \tag{3}$$

We will be particularly focusing on equation 3. It is important to note that the sheaf and the weights in this construction are time-dependent, meaning that the underlying "geometry" evolves over time.

3 Connection Sheaf Laplacians

To construct our sheaf Laplacian $\Delta_{\mathcal{F}(t)}$, we have to construct the individual restriction maps for $\mathcal{F}_{v \trianglelefteq e}$. Instead of learning them via a parametric function $\Phi : \mathbb{R}^{d \times 2} \to \mathbb{R}^{d \times d}$ such that $\mathcal{F}_{v \trianglelefteq e:=(v,u)} = \Phi(\mathbf{x}_v, \mathbf{x}_u)$ as done by Bodnar et al. [3], we approach this problem by computing the restriction maps directly from the data. We choose to do this for orthogonal restriction maps as these have many useful theoretical properties and importantly they are analogous to parallel transport over a manifold.

3.1 Local PCA & Alignment for Point Clouds

A procedure to learn orthogonal transformations between data points was presented in Singer and Wu [24]. The idea stems from the so-called "manifold assumption", the fact that even though data "lives" in a high-dimensional space $\mathbb{R}^p$, the correlation between dimensions suggests that in reality, the data points are distributed on a low dimensional Riemannian manifold [18] $\mathcal{M}^d$ which is embedded in $\mathbb{R}^p$. Here d is the manifold's dimension, and p is the dimension of the embedding space, such that $d \ll p$. First orthonormal bases of the tangent spaces for each data point are constructed via local PCA. Next, the tangent spaces are optimally aligned via orthogonal transformations, which can be thought of as mappings from one tangent space to a neighbouring one. Note the structural similarity with sheaves and, more specifically, figure 2.

We start by describing local PCA which is used to estimate a basis to the tangent space $T_{x_i}\mathcal{M}$ for each point x_i in the point cloud $x_1, \ldots, x_n$. Singer and Wu [24] compute an $\sqrt{\epsilon_{PCA}}$-neighbourhood ball of points for each point x_i denoted $\mathcal{N}_{x_i,\epsilon_{PCA}}$. This forms a set of neighbouring points $x_{i_1}, \ldots, x_{i_{N_i}}$. Then the $p \times N_i$ matrix $X_i = [x_{i_1} - x_i, x_{i_{N_i}} - x_i]$ is constructed, which centers all of the neighbours at x_i. A $N_i \times N_i$ weighting matrix D_i is constructed, giving more importance to neighbours closer to x_i. This allows us to compute the $p \times N_i$ matrix $B_i = X_i D_i$. Then Singular Value Decomposition (SVD) is used on B_i such that $B_i = U_i \Sigma_i V_i^T$. Assuming that the singular values are in decreasing order, the first d left singular vectors are kept (the first d vectors of U_i), forming the matrix O_i. Note that the columns of O_i are orthonormal by construction and they form a d-dimensional subspace of $\mathbb{R}^p$. This basis constitutes our approximation to the basis of the tangent space $T_{x_i}\mathcal{M}$.

To compute the orthogonal matrix O_{ij}, which represents our orthogonal transformation from $T_{x_i}\mathcal{M}$ to $T_{x_j}\mathcal{M}$, it is sufficient to first of all compute the SVD of $O_i^T O_j = U\Sigma V^T$ and then $O_{ij} = UV^T$. O_{ij} is the orthogonal transformation which optimally aligns the tangent spaces $T_{x_i}\mathcal{M}$ and $T_{x_j}\mathcal{M}$ based on their bases O_i and O_j.

3.2 Local PCA & Alignment for Graphs

The aforementioned technique has many valuable theoretical properties but has the limitation that it is designed for point clouds. Instead, we wish to leverage the valuable edge information at our disposal. To do this, instead of computing the neighbourhood $\mathcal{N}_{x_i,\epsilon_{PCA}}$, we take the 1-hop neighborhood $\mathcal{N}^1_{x_i}$ of x_i. The weighting matrix D_i is also problematic as it is computed by considering the p-norm distance between the neighbours. In general, taking p-norm distances assumes that this sort of distance makes sense, which may be true for particular geometries and features, but not for all, although this could be verified experimentally. For the case of a 1-hop neighbourhood, we assume that all nodes are instead weighted the same. For this reason, we abandon the weighting matrix D_i completely and instead set $B_i = X_i$. It may be fruitful to explore the construction of D_i via an attention mechanism, but this is left to future work. After this, the computations are the same. We compute the SVD of B_i to extract O_i from the left singular vectors, and we finally compute O_{ij} from the SVD of $O_i^T O^j$. This gives us a graph-aware version of the technique proposed by Singer and Wu [24], which to the best knowledge of the authors, is a novel technique. A diagram of our newly proposed technique may be seen in figure 3.

One difficulty with this approach is estimating d, the dimension of the tangent space (in our case, the stalks). In fact, we are assuming that every neighbourhood is larger than d or else B_i would have less than d singular vectors, and our construction would be ill-defined. This is clearly not always the case for all d. While Singer and Wu [24] propose to estimate d directly from the data, we leave d as a tunable hyper-parameter. To solve the problem for nodes which have less than d neighbours, we sample an orthogonal matrix at random by constructing a Haar-random matrix (Meckes [19]). However, we emphasize that this is not the only solution. The intuition behind this is that when a node has less than d neighbours, we lack the required information to estimate a d-dimensional restriction map. To circumnavigate this, we sample from a distribution of orthogonal random matrices. One could try to consider an n-hop neighbourhood instead, in a similar fashion to ϵ_{PCA} in the original technique. Still, this comes with a larger computational overhead and complications

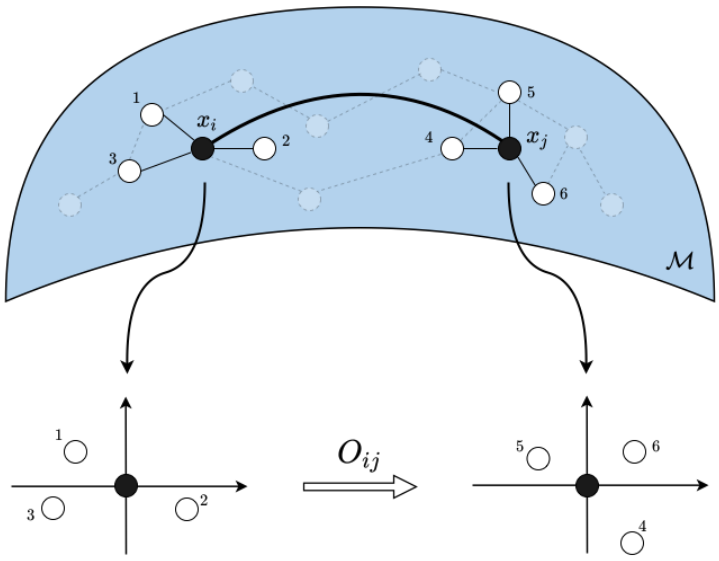

Figure 3: The orthonormal bases of $T_{x_i}\mathcal{M}$ and $T_{x_j}\mathcal{M}$ are determined by local PCA using nodes in the 1-hop neighbourhood of x_i and x_j respectively. The orthogonal mapping O_{ij} is a map from $T_{x_i}\mathcal{M}$ to $T_{x_j}\mathcal{M}$ which optimally aligns their bases.

related to the weightings. Furthermore, if a graph has a disconnected node, this would still be an issue. In practice, d is chosen such that the minority of O_{ij}s need to be randomly sampled.

4 Evaluation

We evaluate our technique with the same models and datasets from Bodnar et al. [3]. These are real-world datasets aimed to evaluate heterophilic learning by Rozemberczki et al. [22], Pei et al. [21]. They are ordered by a homophily coefficient $0 \leq h \leq 1$, which is higher for more homophilic datasets and lower for heterophilic datasets. h is the fraction of edges which connect nodes of the same class label. Due to limited computational resources we do not evaluate on the larger Squirrel dataset. However, we believe this does not detract from the evaluation as the remaining datasets offer a wide range of homophily levels and domains. The results are collected over 10 provided fixed splits, with 48%/32%/20% of nodes for training/validation/testing. The reported results are chosen from the highest validation score to avoid data snooping.

As baselines, we use the same models used in Bodnar et al. [3]. The direct baselines used for comparison are the discrete sheaf diffusion models [3]. On top of these models, we also evaluate a random sheaf baseline with orthogonal Haar-random matrices. The rest of the models may be divided into 4 categories:

1. Classical: GCN (Kipf and Welling [17]), GAT (Velickovic et al. [25]), GraphSAGE (Hamilton et al. [11])
2. Models for heterophilic settings: GGCN (Yan et al. [27]), Geom-GCN (Pei et al. [21]), H2GCN (Zhu et al. [29]), GPRGNN (Chien et al. [8]), FAGCN (Bo et al. [2]), MixHop (Abu-El-Haija et al. [1])
3. Models which address oversmoothing: GCNII (Chen et al. [7]), PairNorm (Zhao and Akoglu [28])
4. Continous models: Continous Sheaf Diffusion (Bodnar et al. [3]), CGNN (Xhonneux et al. [26]), GRAND (Chamberlain et al. [5]), BLEND (Chamberlain et al. [4])

Table 1 shows the results for the aforementioned settings. The Sheaf Diffusion class of models performed very well on the more heterophilic datasets and less so as homophily

	Texas	**Wisconsin**	**Film**	**Chameleon**	**Cornell**	**Citeseer**	**Pubmed**	**Cora**
Hom level	**0.11**	**0.21**	**0.22**	**0.23**	**0.30**	**0.74**	**0.80**	**0.81**
#Nodes	183	251	7,600	2,277	183	3,327	18,717	2,708
#Edges	295	466	26,752	31,421	280	4,676	44,327	5,278
#Classes	5	5	5	5	5	7	3	6
Riem-SD (ours)	**87.16**±2.24	**88.73**±4.47	37.47±0.956	58.64±2.41	**85.95**±7.72	72.61±1.97	87.84±0.44	75.86±2.19
Rand-SD	84.05±5.33	85.69±4.02	37.40±1.18	47.72±1.60	84.59±7.65	72.49±1.91	87.74±0.50	74.00±1.99
Diag-SD	**85.67**±6.95	88.63±2.75	**37.79**±1.01	**68.68**±1.73	**86.49**±7.35	77.14±1.85	89.42±0.43	87.14±1.06
O(d)-SD	**85.95**±5.51	**89.41**±4.74	**37.81**±1.15	**68.04**±1.58	84.86±4.71	76.70±1.57	89.49±0.40	86.90±1.13
Gen-SD	82.97±5.13	**89.21**±3.84	**37.80**±1.22	67.93±1.58	**85.68**±6.51	76.32±1.65	89.33±0.35	87.30±1.15
GGCN	84.86±4.55	86.86±3.29	37.54±1.56	**71.14**±1.84	**85.68**±6.63	77.14±1.45	89.15±0.37	**87.95**±1.05
H2GCN	84.86±7.23	87.65±4.98	35.70±1.00	60.11±2.15	82.70±5.28	77.11±1.57	89.49±0.38	87.87±1.20
GPRGNN	78.38±4.36	82.94±4.21	34.63±1.22	46.58±1.71	80.27±8.11	77.13±1.67	87.54±0.38	**87.95**±1.18
FAGCN	82.43±6.89	82.94±7.95	34.87±1.25	55.22±3.19	79.19±9.79	N/A	N/A	N/A
MixHop	77.84±7.73	75.88±4.90	32.22±2.34	60.50±2.53	73.51±6.34	76.26±1.33	85.31±0.61	87.61±0.85
GCNII	77.57±3.83	80.39±3.40	37.44±1.30	63.86±3.04	77.86±3.79	**77.33**±1.48	**90.15**±0.43	**88.37**±1.25
Geom-GCN	66.76±2.72	64.51±3.66	31.59±1.15	60.00±2.81	60.54±3.67	**78.02**±1.15	**89.95**±0.47	85.35±1.57
PairNorm	60.27±4.34	48.43±6.14	27.40±1.24	62.74±2.82	58.92±3.15	73.59±1.47	87.53±0.44	85.79±1.01
GraphSAGE	82.43±6.14	81.18±5.56	34.23±0.99	58.73±1.68	75.95±5.01	76.04±1.30	88.45±0.50	86.90±1.04
GCN	55.14±5.16	51.76±3.06	27.32±1.10	64.82±2.24	60.54±5.30	76.50±1.36	88.42±0.50	86.98±1.27
GAT	52.16±6.63	49.41±4.09	27.44±0.89	60.26±2.50	61.89±5.05	76.55±1.23	87.30±1.10	86.33±0.48
MLP	80.81±4.75	85.29±3.31	36.53±0.70	46.21±2.99	81.89±6.40	74.02±1.90	75.69±2.00	87.16±0.37
Cont Diag-SD	82.97±4.37	86.47±2.55	36.85±1.21	62.06±3.84	80.00±6.07	76.56±1.19	89.47±0.42	86.88±1.21
Cont O(d)-SD	82.43±5.95	84.50±4.34	36.39±1.37	63.18±1.69	72.16±10.40	75.19±1.67	89.12±0.30	86.70±1.24
Cont Gen-SD	83.78±6.62	85.29±3.31	37.28±0.74	66.40±2.28	84.60±4.69	**77.54**±1.72	**89.67**±0.40	87.45±0.99
BLEND	83.24±4.65	84.12±3.56	35.63±0.89	60.11±2.09	**85.95**±6.82	76.63±1.60	89.24±0.42	**88.09**±1.22
GRAND	75.68±7.25	79.41±3.64	35.62±1.01	54.67±2.54	82.16±7.09	76.46±1.77	89.02±0.51	87.36±0.96
CGNN	71.35±4.05	74.31±7.26	35.95±0.86	46.89±1.66	66.22±7.69	76.91±1.81	87.70±0.49	87.10±1.35

Table 1: Results for node classification datasets. The datasets are sorted by increasing order of homophily. Our technique is denoted Riem-SD, while the other Sheaf Diffusion models are Diag-SD, O(d)-SD and Gen-SD. The top three models are coloured by **First**, **Second** and **Third** respectively. The first section contains the discrete models, while the second the continuous ones.

increased. Our technique (Riem-SD) showed promising results, with the highest performance on the Texas dataset, second-best on Cornell and third-best on Wisconsin. It also performed competitively on Film and Pubmed. Unfortunately, on Chameleon, Citeseer and Cora, the performance was not as impressive. Riem-SD, alongside the other original discrete sheaf diffusion methods (Orig-SD), consistently beat the random orthogonal sheaf baseline. Importantly, on Chameleon, Riem-SD performs significantly better than the underlying MLP, while this is not the case for the Rand-SD model. This is a promising result because, in Chameleon, the underlying MLP performs significantly worse than the top-scoring models, while this discrepancy is less the case for the other datasets.

It is critical to mention that due to the limited computational resources, fine-tuning for Riem-SD was very minimal. Instead, this was not the case for the original sheaf diffusion methods. For Riem-SD, we only ran 10 fine-tuning experiments for the smaller datasets and for Pubmed and Chameleon we only ran 3. We chose the hyper-parameter settings based on the highest-scoring validation fine-tuning runs of the O(d)-SD model on the respective datasets. In comparison for the O(d)-SD model alone, ignoring the Squirrel dataset, Bodnar et al. [3] ran $2,359$ fine-tuning runs for the reported results. Choosing the highest validation accuracy scoring settings allowed us to search the hyper-parameter space more efficiently. Still, this carries unfavourable bias over from O(d)-SD as these settings worked well for a different model. We, therefore, expect more fine-tuning to achieve even better results.

One explanation for the success of Riem-SD is that it tackles a problem which comes with discrete sheaf diffusion, which is that of learning the parameters of the sheaf. Especially for the general restriction map case, the sheaf Laplacian is parametrised by many parameters, which makes it more prone to overfitting. The diagonal and orthogonal restriction maps reduce the number of parameters, but it remains a significant increase. Instead, with our technique, the Laplacian is not learnt but simply computed and remains fixed throughout training, removing many parameters from the model. While it is true that the model becomes less expressive, the advantage is that the diffusion becomes less prone to overfitting. This explains why Riem-SD performs best on the smaller datasets: Texas, Wisconsin and Cornell.

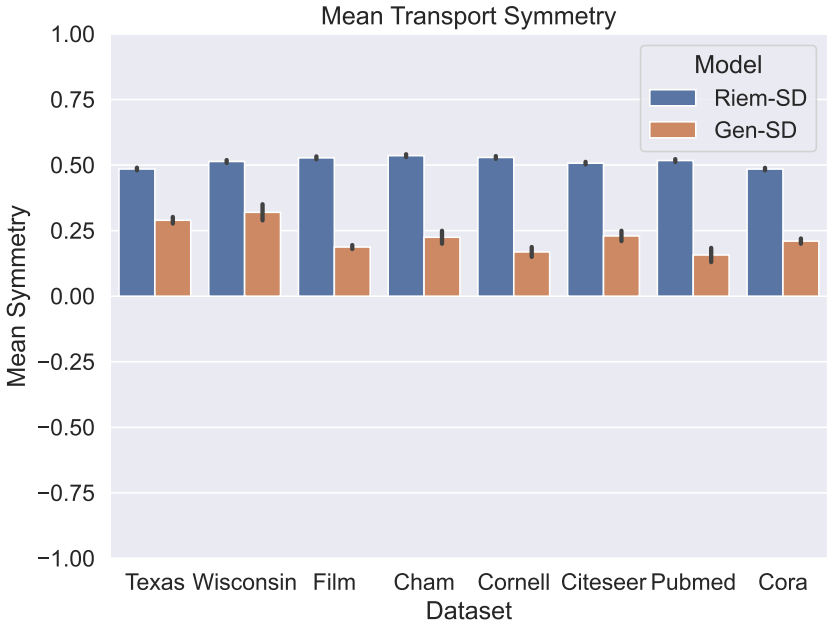

Figure 4: The mean transport symmetry computed for all the maps $\mathcal{F}_{v\trianglelefteq u}^T \mathcal{F}_{v\trianglelefteq u}$, where the symmetry score for a matrix $\mathbf{A}$ is computed as $\text{sym}(\mathbf{A}) = \frac{||\mathbf{A}_{\text{sym}}||-||\mathbf{A}_{\text{anti-sym}}||}{||\mathbf{A}_{\text{sym}}||+||\mathbf{A}_{\text{anti-sym}}||}$. The mean symmetries are computed over 10 folds. The Gen-SD model is shown for comparison to the Riem-SD model. The datasets are ordered in increasing levels of homophily from left to right. The variance in Riem-SD within a dataset is given by the Haar-random matrices.

4.1 Restriction Map Symmetry

The lower performance on some of the datasets may be partly due to the low amount of fine-tuning but could hint at a deeper problem with how the sheaf Laplacian is being computed. It may be the case that the type of orthogonal maps computed are not well-suited for certain types of datasets. While, instead, learning the sheaf may give extra flexibility and more expressive power. For example, we may want to learn maps which are more or less symmetric depending on the homophily levels of the dataset. In fact, Bodnar et al. [3] show that asymmetric restriction maps are needed for heterophilic data. Intuitively, one may think that when the restriction maps are symmetric (or close to symmetric), the two nodes contribute to each other in a symmetric manner, which is a desired property for homophilic datasets.

To evaluate this flexibility, we plot the mean symmetry of the restriction maps against the homophily coefficient h. To compute the mean symmetry, we first compute all of the maps $\mathcal{F}_{v\trianglelefteq u}^T \mathcal{F}_{v\trianglelefteq u}$ for nodes v and u which share an edge e and then compute their symmetry measure. To compute the symmetry measure for a matrix $\mathbf{A}$, we compute its symmetric component $\mathbf{A}_{\text{sym}} = 0.5(\mathbf{A}+\mathbf{A}^T)$ and its anti-symmetric component $\mathbf{A}_{\text{anti-sym}} = 0.5(\mathbf{A}-\mathbf{A}^T)$. We then compute the score $\text{sym}(\mathbf{A}) = \frac{||\mathbf{A}_{\text{sym}}||-||\mathbf{A}_{\text{anti-sym}}||}{||\mathbf{A}_{\text{sym}}||+||\mathbf{A}_{\text{anti-sym}}||}$ where with $||\mathbf{A}||$ we mean the matrix norm of $\mathbf{A}$, which in our case is the matrix norm induced by the vector p-norm with $p = 2$. The score $\text{sym}(\mathbf{A})$ will be 1 when the matrix $\mathbf{A}$ is totally symmetric and -1 when the matrix is totally anti-symmetric.

In figure 4, we report the mean of this value over all of the edges for the datasets over 10 folds. For Riem-SD, the mean symmetry has low variance between the datasets and does not seem affected by homophily levels. This is not unexpected, as the homophily coefficient is computed based on the labels of the nodes, which is information which is not provided when computing the maps O_{ij}. To circumnavigate this, it may be useful to train an orthogonal sheaf diffusion layer to predict the pre-computed Riemannian sheaf and then

allow the layer to train as usual. This would reap the benefits from the principled structure of the pre-computed sheaf, alongside the advantages which come from tuning the sheaf via gradient-based methods.

In figure 4, we also notice how the mean symmetry of the Gen-SD model, with general linear maps, does not increase as the datasets become more homophilic. In fact, it seems to decrease. However, the mean symmetry has a lot more variance throughout the different datasets. The extra variance may explain why learning the sheaf achieves better performance, as the model adapts the sheaf more heavily depending on the dataset. The fact that the symmetry does not increase as the homophily increases may explain why the original sheaf diffusion models do not perform as well on the more homophilic datasets such as Citeseer, Pubmed and Cora as they do on the Heterophilic datasets. It may be useful to explore this issue further as fixing this behaviour may make sheaf diffusion models perform better in homophilic settings.

5 Conclusion

This work proposes a novel technique to compute the sheaf Laplacian of a graph deterministically. This was done by leveraging existing differential geometry work that constructs orthogonal maps that optimally align tangent spaces between points, relying on the manifold assumption. We crucially adapted this intuition to be graph-aware, leveraging the vital edge connections in the graph structure.

We showed that this technique achieves promising empirical results. With very little fine-tuning, we outperformed the performance of the baselines sheaf diffusion models on some data sets and obtained competitive performance on others. We also provide empirical evidence related to the symmetry of the learnt sheaves, unveiling potential issues of the newly proposed technique and the previous sheaf diffusion models, alongside possible solutions.

We believe in having uncovered an exciting direction of research which aims to find a way to compute a sheaf without a gradient-based approach. Furthermore, we are excited by the prospect of further research aiming to tie intuition from the fields of algebraic topology and differential geometry to machine learning. We believe that this work forms a promising first step in this direction.

6 Acknowledgments

We want to thank the amazing Cristian Bodnar for the countless helpful discussions and explanations and being such a wonderful and available supervisor. We also would like to thank the lovely girlfriend of the first author for allowing the experiments to be run on her shiny GPU, even if it meant she had to play Valorant on her work laptop instead.[2] Of course, a warm thank you to the L45 staff for organizing such a challenging, exciting and instructive module and a special thank you to Pietro and Petar for helping with a much-needed extension for the MLMI students.

References

[1] S. Abu-El-Haija, B. Perozzi, A. Kapoor, N. Alipourfard, K. Lerman, H. Harutyunyan, G. Ver Steeg, and A. Galstyan. Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing. In *international conference on machine learning*, pages 21–29. PMLR, 2019.

[2] D. Bo, X. Wang, C. Shi, and H. Shen. Beyond low-frequency information in graph convolutional networks. *arXiv preprint arXiv:2101.00797*, 2021.

[3] C. Bodnar, F. Di Giovanni, B. P. Chamberlain, P. Lio, and M. M. Bronstein. Neural sheaf diffusion: A topological perspective on heterophily and oversmoothing in gnns. In *ICLR 2022 Workshop on Geometrical and Topological Representation Learning*, 2022.

[2] Unfortunately, the first author has developed a profound hatred for Google Colab after all of the session time-outs.

[4] B. Chamberlain, J. Rowbottom, D. Eynard, F. Di Giovanni, X. Dong, and M. Bronstein. Beltrami flow and neural diffusion on graphs. *Advances in Neural Information Processing Systems*, 34, 2021.

[5] B. Chamberlain, J. Rowbottom, M. I. Gorinova, M. Bronstein, S. Webb, and E. Rossi. Grand: Graph neural diffusion. In *International Conference on Machine Learning*, pages 1407–1418. PMLR, 2021.

[6] D. Chen, Y. Lin, W. Li, P. Li, J. Zhou, and X. Sun. Measuring and relieving the over-smoothing problem for graph neural networks from the topological view. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 3438–3445, 2020.

[7] M. Chen, Z. Wei, Z. Huang, B. Ding, and Y. Li. Simple and deep graph convolutional networks. In *International Conference on Machine Learning*, pages 1725–1735. PMLR, 2020.

[8] E. Chien, J. Peng, P. Li, and O. Milenkovic. Joint adaptive feature smoothing and topology extraction via generalized pagerank gnns. *arXiv preprint arXiv:2006.07988*, 2020.

[9] J. M. Curry. *Sheaves, cosheaves and applications*. University of Pennsylvania, 2014.

[10] A. Davies, P. Veličković, L. Buesing, S. Blackwell, D. Zheng, N. Tomašev, R. Tanburn, P. Battaglia, C. Blundell, A. Juhász, et al. Advancing mathematics by guiding human intuition with ai. *Nature*, 600(7887):70–74, 2021.

[11] W. Hamilton, Z. Ying, and J. Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.

[12] J. Hansen. *Laplacians of Cellular Sheaves: Theory and Applications*. PhD thesis, University of Pennsylvania, 2020.

[13] J. Hansen and T. Gebhart. Sheaf neural networks. *arXiv preprint arXiv:2012.06333*, 2020.

[14] J. Hansen and R. Ghrist. Toward a spectral theory of cellular sheaves. *Journal of Applied and Computational Topology*, 3(4):315–358, 2019.

[15] J. Hansen and R. Ghrist. Opinion dynamics on discourse sheaves. *SIAM Journal on Applied Mathematics*, 81(5):2033–2060, 2021.

[16] A. Hatcher. *Algebraic topology*. , 2005.

[17] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

[18] J. M. Lee. *Riemannian manifolds: an introduction to curvature*, volume 176. Springer Science & Business Media, 2006.

[19] E. S. Meckes. *The random matrix theory of the classical compact groups*, volume 218. Cambridge University Press, 2019.

[20] K. Oono and T. Suzuki. Graph neural networks exponentially lose expressive power for node classification. *arXiv preprint arXiv:1905.10947*, 2019.

[21] H. Pei, B. Wei, K. C.-C. Chang, Y. Lei, and B. Yang. Geom-gcn: Geometric graph convolutional networks. *arXiv preprint arXiv:2002.05287*, 2020.

[22] B. Rozemberczki, C. Allen, and R. Sarkar. Multi-scale attributed node embedding. *Journal of Complex Networks*, 9(2):cnab014, 2021.

[23] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini. The graph neural network model. *IEEE transactions on neural networks*, 20(1):61–80, 2008.

[24] A. Singer and H.-T. Wu. Vector diffusion maps and the connection laplacian. *Communications on pure and applied mathematics*, 65(8):1067–1144, 2012.

[25] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio. Graph attention networks. *stat*, 1050:20, 2017.

[26] L.-P. Xhonneux, M. Qu, and J. Tang. Continuous graph neural networks. In *International Conference on Machine Learning*, pages 10432–10441. PMLR, 2020.

[27] Y. Yan, M. Hashemi, K. Swersky, Y. Yang, and D. Koutra. Two sides of the same coin: Heterophily and oversmoothing in graph convolutional neural networks. *arXiv preprint arXiv:2102.06462*, 2021.

[28] L. Zhao and L. Akoglu. Pairnorm: Tackling oversmoothing in gnns. *arXiv preprint arXiv:1909.12223*, 2019.

[29] J. Zhu, Y. Yan, L. Zhao, M. Heimann, L. Akoglu, and D. Koutra. Generalizing graph neural networks beyond homophily. *arXiv preprint arXiv:2006.11468*, 2020.